

The Control Plane in Action

Step-by-Step: Dijkstra's Link-State Algorithm

Bernal Manzanilla

Concordia University of Edmonton
IT 213: Networking

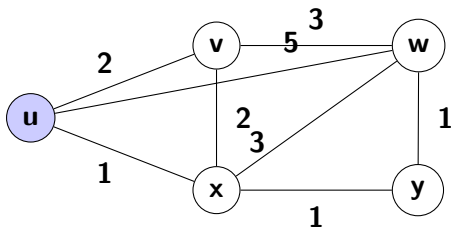
Winter 2026

Recap: The Control Plane and Link-State

- **The Goal:** The Control Plane's job is to compute the least-cost path from a source router to all other routers in the network.
- **Link-State (LS):** In an LS algorithm like OSPF, every router broadcasts its link costs to all other routers. As a result, every router has an identical, complete map of the network (the Adjacency Matrix).
- **Dijkstra's Algorithm:** With this complete map, the router runs Dijkstra's algorithm to compute the shortest-path tree.
- **Terminology:**
 - N' : The set of nodes whose least-cost path is definitively known.
 - $D(v)$: The current cost of the path from the source to node v .
 - $p(v)$: The previous node (predecessor) along the current shortest path to v .
 - $c(x, y)$: The cost of the direct physical link between node x and node y .

Our Network Topology

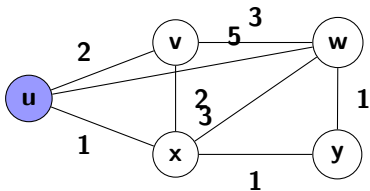
Let's trace the algorithm from the perspective of our source router, **Node u**.



- We will maintain a table tracking N' and the current costs D and predecessors p for all other nodes.

Step 0: Initialization

- Logic:** We start by adding our source node u to the known set N' . We then look at the direct physical links from u to its neighbors. If there is no direct link, the cost is infinity (∞).



Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$
0	$\{u\}$	2, u	5, u	1, u	∞

Step 1: First Iteration

- **Logic:** Find the node not in N' with the lowest cost. Here, $D(x) = 1$ is the minimum. We add x to N' .
- We now update the paths to x 's neighbors (v, w, y) using the formula: $D(v) = \min(D(v), D(x) + c(x, v))$.
- $D(v) = \min(2, 1 + 2) = 2$. No change.
- $D(w) = \min(5, 1 + 3) = 4$. **Updated!** New predecessor is x .
- $D(y) = \min(\infty, 1 + 1) = 2$. **Updated!** New predecessor is x .

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$
0	$\{u\}$	2, u	5, u	1, u	∞
1	$\{u, x\}$	2, u	4, x		2, x

Step 2: Second Iteration

- **Logic:** Find the next node not in N' with the lowest cost. Nodes v and y both have a cost of 2. We can pick either. Let's pick y and add it to N' .
- We update y 's neighbor (w): $D(w) = \min(D(w), D(y) + c(y, w))$.
- $D(w) = \min(4, 2 + 1) = 3$. **Updated!** We found a cheaper path to w by going through y . The new predecessor is y .

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$
0	$\{u\}$	2, u	5, u	1, u	∞
1	$\{u, x\}$	2, u	4, x		2, x
2	$\{u, x, y\}$	2, u	3, y		

Step 3: Third Iteration

- **Logic:** Find the next minimum cost node not in N' . Node v has a cost of 2, while node w has a cost of 3. We pick v and add it to N' .
- Update v 's neighbor (w): $D(w) = \min(D(w), D(v) + c(v, w))$.
- $D(w) = \min(3, 2 + 3) = 3$. No change. The path through y is still cheaper.

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$
0	$\{u\}$	2, u	5, u	1, u	∞
1	$\{u, x\}$	2, u	4, x		2, x
2	$\{u, x, y\}$	2, u	3, y		
3	$\{u, x, y, \mathbf{v}\}$		3, y		

Step 4: Final Iteration

- **Logic:** Only node w remains. We add w to N' . The algorithm terminates because N' now contains all nodes in the network.

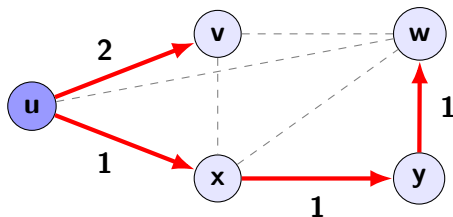
Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$
0	$\{u\}$	2, u	5, u	1, u	∞
1	$\{u, x\}$	2, u	4, x		2, x
2	$\{u, x, y\}$	2, u	3, y		
3	$\{u, x, y, v\}$		3, y		
4	$\{u, x, y, v, w\}$				

From Algorithm to Forwarding Table

By tracing the predecessors $p(v)$ backward from destinations to u , router u constructs its shortest-path tree. It then installs the **first hop** of these paths into its hardware Forwarding Table (the Data Plane)!

The Final Shortest-Path Tree

Here is the final tree computed by router u . It highlights the optimal links chosen by Dijkstra's algorithm.



- **Forwarding Table at u :**

- Dest $v \rightarrow$ Link (u, v)
- Dest $x \rightarrow$ Link (u, x)
- Dest $y \rightarrow$ Link (u, x)
- Dest $w \rightarrow$ Link (u, x)